

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-aa4b0d05510f6bf5a.jsonl`  
- SHA-256: `f434f523dbd3ae936d70b3e5a58632a6eb41c6a7b69fbe554f9330627048ed3c`  
- Source modified: `2026-07-06T19:19:01+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:17:45.565Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (claims): F1 'wire into live-message handler' understates the work: the handler uses VideoRecord/insert_video and FilterEngine.evaluate(VideoMeta) ? none of PolicyContext's inputs (ContentItem, Source, Policy) are on hand
severity medium | section Implementation phasing ? F1 (dependencies: None); Stage interface ? PolicyStage / PolicyContext
summary: PolicyContext requires content_item: ContentItem, source: Source, and policy: Policy. The current live handler operates on a VideoRecord + VideoMeta and calls
FilterEngine.evaluate(meta: VideoMeta); it does not construct a ContentItem, does not load a Source object, and does not load a Policy for the account. F1 claims dependency 'None' and frames the wiring as extending 'today's FilterEngine', but assembling a PolicyContext in the handler is non-trivial and unstated.
why_it_matters: Marking F1 as depending on 'None' misrepresents the integration cost and hides that F1 must first migrate the handler from the legacy VideoRecord/insert_video path to the ContentItem/Source/Policy path (or build adapters). This is the highest-risk part of Track F and the phase table gives it no visibility.
evidence: telethon_client.py:257 result = engine_holder[ENGINE_KEY].evaluate(meta); :258-269 builds VideoRecord; :271 insert_video. FilterEngine.evaluate signature is (self, meta: VideoMeta) with VideoMeta fields = duration_s,width,height,size_bytes,mime_type,is_gif,is_round,nosound (no ContentItem/Source/Policy). PolicyContext (ADR) needs ContentItem+Source+Policy. Tier 1 'extends today's FilterEngine' but FilterEngine's input is VideoMeta, not PolicyContext.
rec: Note in F1 that wiring the pipeline into the live handler requires the handler to first produce a ContentItem + resolve the owning Source + load the account Policy (today it produces a VideoRecord and evaluates VideoMeta). Either add that migration as an explicit F1 sub-task/dependency or state that F1 adapts VideoMeta->PolicyContext, so 'dependencies: None' is not misread as 'drops into the existing handler unchanged'.

2. assistant (2026-07-06T19:17:50.284Z)

I'll verify this finding by examining the ADR and the current code.

3. user (2026-07-06T19:17:52.173Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

```
| Concept | Current state |
|---|---|
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude
rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig`
blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were
deferred to v2 in the original plan. |
```

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                               Policy Pipeline                       ?
?                               ?                                     ?
? Tier 1 ? Metadata gates (always runs, zero I/O)                   ?
?   duration, resolution, size, codec, mime, GIF/round              ?
?   ? extends today's FilterEngine                                  ?
?                               ?                                     ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)                ?
?   source allowlist, keyword match, channel topic tags            ?
?   ? user-expressed interests filter the stream                   ?
?                               ?                                     ?
? Tier 3 ? Content safety (opt-in, metadata + light I/O)            ?
?   URL/domain reputation, file metadata sanity,                   ?
?   caption text scanning                                           ?
?   ? cheap checks; no download                                     ?
?                               ?                                     ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)                      ?
?   ffprobe, perceptual hash, downloaded content scan              ?
?   ? gated behind explicit user opt-in + budget                   ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
```

```

 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
...

```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```

```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated** behind explicit opt-in because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
```
```

The caller (telethon\_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

## Schema ? minimal changes

**\*\*No schema migration\*\*** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

## Implementation phasing (future issues)

| Phase   | Scope                   | Dependencies                                                                  |
|---------|-------------------------|-------------------------------------------------------------------------------|
| ***F1** | Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler |
| ***F2** | Topic preferences       | `topics` config parsing, source title/caption keyword matching                |
| ***F3** | Source allowlist        | `allowlist` config, skip non-allowlisted sources                              |
| ***F4** | URL/domain reputation   | Domain blacklist, caption link extraction                                     |
| ***F5** | Deep inspection         | ffprobe/perceptual hash, download budget enforcement                          |
| ***F6** | Policy decisions table  | Audit trail storage, explainability UI                                        |
| ***F7** | Explainability endpoint | `/why <content_item_id>` command showing the explanation trace                |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the **\*\*design contract\*\*** the phases code against.

## Consequences

### **\*\*Positive\*\***

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

## **\*\*Negative\*\***

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

## **Alternatives considered**

- **\*\*Do nothing; keep flat FilterConfig.\*\*** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- **\*\*Inline each stage in the handler\*\*** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- **\*\*Use a rules engine / DSL.\*\*** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- **\*\*Bundle all phases into a single PR.\*\*** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

## **Open questions for the decision owner**

1. **\*\*`StageResult.metadata` shape\*\*** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. **\*\*Policy decisions table\*\*** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. **\*\*Topic matching\*\*** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. **\*\*Deep inspection budget\*\*** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

## **References**

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

### **4. assistant (2026-07-06T19:17:55.237Z)**

Now let me examine the current handler code and models to verify the finding's claims.

### **5. user (2026-07-06T19:17:57.553Z)**

c4c4d18 fix(scanning): don't clobber user-controlled watched on source upsert (#41)  
3bfb1da feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR 0009, Track E) (#39)  
35f4f70 feat(commands): extract CommandService ? provider-independent command execution (ADR

**6. user (2026-07-06T19:17:58.211Z)**

```

 if existing is not None:
 log.info(
 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
 "channel=%s msg=%s original_channel=%s original_msg=%s",
 user.id,
 meta.size_bytes,
 meta.duration_s,
 meta.height,
 meta.width,
 channel_id,
 message_id,
 existing.channel_id,
 existing.message_id,
)
 return

Lazily record the channel (best-effort; failures are non-fatal).
try:
 await db.upsert_channel(
 conn,
 Channel(user_id=user.id, channel_id=channel_id, title=_channel_title(event)),
)
except Exception:
 log.warning(
 "failed to upsert channel %s for user %s", channel_id, user.id, exc_info=True
)

result = engine_holder[ENGINE_KEY].evaluate(meta)
record = VideoRecord(
 user_id=user.id,
 channel_id=channel_id,
 message_id=message_id,
 document_id=document_id,
 duration_s=meta.duration_s,
 width=meta.width,
 height=meta.height,
 size_bytes=meta.size_bytes,
 mime_type=meta.mime_type,
 matched=result.matched,
)
try:
 inserted = await db.insert_video(conn, record)
except sqlite3.IntegrityError:
 # Lost a dedup race: a concurrent delivery of the same file
 # (same document_id or same message instance) won the insert.
 # The three-tier check above already classified this as new, so
 # the collision means another task is processing (or has
 # processed) it ? log and skip.
 log.info(
 "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
 user.id,
 channel_id,
 message_id,
 document_id,
)
 return
log.info(
 "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
 user.id,

```

```
channel_id,
message_id,
```

## 7. assistant (2026-07-06T19:18:02.147Z)

The finding cites `evaluate(meta)` and VideoRecord ? confirmed. Now let me check the FilterEngine.evaluate signature and the models (ContentItem, Source, Policy, VideoMeta).

## 8. user (2026-07-06T19:18:03.767Z)

(Bash completed with no output)

## 9. user (2026-07-06T19:18:04.885Z)

```
123:class Source(BaseModel):
149:class ContentItem(BaseModel):
186:class Policy(BaseModel):
299:class FilterConfig(BaseModel):
383:class VideoRecord(BaseModel):
```

## 10. user (2026-07-06T19:18:09.705Z)

```
src/tg_video_filter/filters/base.py:15:class VideoMeta(BaseModel):
src/tg_video_filter/filters/base.py:34:class FilterResult:
src/tg_video_filter/filters/base.py:63: def evaluate(self, meta: VideoMeta) -> FilterResult:
src/tg_video_filter/filters/content_type.py:17: def evaluate(self, meta: VideoMeta) ->
FilterResult:
src/tg_video_filter/filters/content_type.py:28: def evaluate(self, meta: VideoMeta) ->
FilterResult:
src/tg_video_filter/filters/content_type.py:39: def evaluate(self, meta: VideoMeta) ->
FilterResult:
src/tg_video_filter/filters/duration.py:20: def evaluate(self, meta: VideoMeta) ->
FilterResult:
src/tg_video_filter/filters/resolution.py:18: def evaluate(self, meta: VideoMeta) ->
FilterResult:
src/tg_video_filter/filters/size.py:20: def evaluate(self, meta: VideoMeta) -> FilterResult:
src/tg_video_filter/filters/__init__.py:28:class FilterEngine:
src/tg_video_filter/filters/__init__.py:35: def evaluate(self, meta: VideoMeta) ->
FilterResult:
```

## 11. user (2026-07-06T19:18:17.865Z)

```
class VideoMeta(BaseModel):
 """Metadata extracted from a Telegram video message.

 All fields are optional because Telegram metadata can be partial; filters
 decide how to treat ``None`` (by default: treat as a non-match with a
 descriptive reason).
 """

 duration_s: int | None = None
 width: int | None = None
 height: int | None = None
 size_bytes: int | None = None
 mime_type: str | None = None
 is_gif: bool | None = None
 is_round: bool | None = None
 nosound: bool | None = None

@dataclass(frozen=True)
class FilterResult:
 """Outcome of evaluating one (or many) filter(s) against a ``VideoMeta``.
```

```

Attributes:
 matched: True if the video passes the filter(s).
 reasons: Human-readable strings, one per evaluated rule. For an AND
 engine this includes every evaluated rule; empty if no rules.
"""

matched: bool
reasons: list[str] = field(default_factory=list)

@staticmethod
def pass_(reason: str) -> FilterResult:
 return FilterResult(matched=True, reasons=[reason])

@staticmethod
def fail(reason: str) -> FilterResult:
 return FilterResult(matched=False, reasons=[reason])

class Filter(ABC):
 """Abstract base for a single filter rule.

 A rule that cannot be evaluated (e.g. ``duration_s is None``) must return
 ``matched=False`` with a reason explaining the missing data.
 """
 ===ENGINE===
class FilterEngine:
 """Evaluates a VideoMeta against a set of rules with AND semantics."""

 def __init__(self, cfg: FilterConfig) -> None:
 self.cfg = cfg
 self.rules: list[Filter] = _build_rules(cfg)

 def evaluate(self, meta: VideoMeta) -> FilterResult:
 """Return the aggregated result of all rules.

 AND semantics: a video matches only if every rule passes. ``reasons``
 contains one entry per evaluated rule (pass or fail). If no rules are
 configured, the video matches vacuously with an empty reason list.
 """
 if not self.rules:
 return FilterResult(matched=True, reasons=[])

 all_matched = True
 reasons: list[str] = []
 for rule in self.rules:
 r = rule.evaluate(meta)
 reasons.extend(r.reasons)
 if not r.matched:
 all_matched = False
 return FilterResult(matched=all_matched, reasons=reasons)

def _build_rules(cfg: FilterConfig) -> list[Filter]:
 """Build the rule list, skipping rules whose bound is None.

 (ResolutionFilter/SizeFilter also defend against None internally, but
 omitting them here keeps the reasons list free of "rule disabled" noise and
 makes the AND short-circuit semantics explicit.)

```

## 12. user (2026-07-06T19:18:18.372Z)

```

class Source(BaseModel):
 """A configured place we ingest content from (ADR 0011).

 Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future

```

```
providers add their own source kinds). ``provider_source_id`` is the
provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
"""
```

```
model_config = ConfigDict(frozen=True)
```

```
id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
account_id: int = Field(description="FK ? accounts.id (owner of the source)")
provider: str = Field(default="telegram", description="Provider namespace")
provider_source_id: str = Field(
 description="Provider-native source id (Telegram channel_id, ?)"
)
title: str | None = None
kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
watched: bool = Field(
 default=False,
 description="ADR 0009 selective scanning: ingest from this source iff True",
)
seen_first_at: datetime = Field(default_factory=_utcnow)
last_seen_at: datetime | None = None
```

```
class ContentItem(BaseModel):
```

```
 """A normalized content record discovered from any :class:`Source` (ADR 0011).
```

```
 Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
 (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
 NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
 ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
 wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
 """
```

```
model_config = ConfigDict(frozen=True)
```

```
id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
account_id: int = Field(description="FK ? accounts.id")
source_id: int = Field(description="FK ? sources.id")
provider: str = Field(default="telegram", description="Provider namespace")
provider_message_id: str = Field(
 description="Provider-native message id (Telegram message_id, ?)"
)
content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
text: str | None = Field(default=None, description="Caption / post text (nullable in Phase
1)")
link_url: str | None = Field(default=None, description="For link/article posts (Phase 1:
NULL)")
media_ref: str | None = Field(default=None, description="Provider media reference")
document_id: str | None = Field(
 default=None,
 description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
)
duration_s: int | None = None
width: int | None = None
height: int | None = None
size_bytes: int | None = None
mime_type: str | None = None
dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique per
account")
matched: bool = False
seen_at: datetime = Field(default_factory=_utcnow)
```

```
class Policy(BaseModel):
```

```
 """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
```

```

Generalizes the legacy per-account ``filter_config`` row so one account can
own multiple policies (enables ADR 0009's per-route filter overrides) and so
future enrichment/security stages grow into the same ``config_json`` column.
Exactly one policy per account carries ``is_default=True``; the legacy
``load_filter_config`` / ``save_filter_config`` shims read/write that row.
"""

model_config = ConfigDict(frozen=True)

id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
account_id: int = Field(description="FK ? accounts.id")
name: str = Field(description="Policy name; 'default' for the account default policy")
config_json: str = Field(
 description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
)
is_default: bool = Field(
 default=False, description="Exactly one per account (UNIQUE partial index)"
)
created_at: datetime = Field(default_factory=utcnow)
updated_at: datetime = Field(default_factory=utcnow)

```

### --- Track C: destinations, routing, delivery state (ADR 0011) -----

#  
**These tables were created empty by Track B (\_SCHEMA); Track C populates destinations (backfilled from filter\_config\_deprecated.config\_json), writes delivery\_attempts (replacing the legacy videos.forwarded boolean), and defines the models for per-destination delivery state. The source/destination invariant (a channel is either source or destination, never both) is enforced at the app level by future selective-scanning logic (Track E, #30).**

```

class Destination(BaseModel):
 """A delivery target ? where matched content is sent (ADR 0011, Track C).

 Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
 can have multiple named output channels. Backfilled from the existing
 per-account ``delivery_target``; the default destination is the target used
 when no explicit routing rule matches.
 """

 model_config = ConfigDict(frozen=True)

```

#### 13. assistant (2026-07-06T19:18:21.581Z)

All claims verified. Let me confirm the handler does NOT construct any of the PolicyContext inputs (ContentItem, Source object, Policy). Let me check whether the handler loads a Source or Policy object anywhere.

#### 14. user (2026-07-06T19:18:25.157Z)

```

12: ``Message`` and returning a ``VideoMeta | None``. This isolates the only piece
37: from tg_video_filter.filters import FilterEngine, VideoMeta
38: from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User, VideoRecord
46: def extract_video_meta(message: Message) -> VideoMeta | None:
47: """Extract ``VideoMeta`` from a Telethon ``Message``, or ``None`` if not a video.
50: ``VideoMeta`` (with fields possibly ``None``) for a video whose individual
79: meta = VideoMeta(
158: 1. extracts VideoMeta + document_id (skips if not a video),

```

```

163: 4. inserts a VideoRecord row,
189: if not await db.should_ingest_source(conn, user.id, channel_id):
258: record = VideoRecord(

```

#### 15. assistant (2026-07-06T19:18:31.532Z)

Confirmed. The handler imports only `Channel`, `DeliveryMode`, `TelegramSession`, `User`, `VideoRecord` from models ? never `ContentItem`, `Source`, or `Policy`. It operates entirely on the legacy `VideoRecord`/`VideoMeta` path. Let me verify the `should\_ingest\_source` gate uses a raw `channel\_id` rather than a `Source` object.

#### 16. user (2026-07-06T19:18:33.236Z)

```

1168:async def insert_content_item(conn: aiomysqlite.Connection, item: ContentItem) ->
ContentItem:
1572:async def should_ingest_source(
1601:async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:

```

#### 17. user (2026-07-06T19:18:36.315Z)

```

async def should_ingest_source(
 conn: aiomysqlite.Connection, account_id: int, channel_id: int
) -> bool:
 """Return True if a channel should be ingested (ADR 0009 opt-in gate).

 - Absent sources row: create it lazily with ``watched=False`` for
 ``/channels`` discoverability; return False (not yet watched).
 - Existing row: respect the ``watched`` flag.

 Pure helper extracted from ``telethon_client.py`` for testability.
 """
 src = await get_source_by_provider_id(conn, account_id, "telegram", str(channel_id))
 if src is None:
 # First-ever message from this channel ? create a discoverable
 # unwatched row so /channels can list it (ADR 0009 lines 48-54).
 await upsert_source(
 conn,
 Source(
 account_id=account_id,
 provider="telegram",
 provider_source_id=str(channel_id),
 kind="channel",
 watched=False,
),
)
 return False
 return src.watched

```

#### 18. assistant (2026-07-06T19:18:49.281Z)

This confirms it. The gate `should\_ingest\_source(conn, account\_id, channel\_id)` takes raw ints and internally loads the `Source`, but the returned `Source` object is discarded (returns only `src.watched`). The handler never holds a `Source`, `ContentItem`, or `Policy` object.

All evidence for the finding checks out:

1. **Handler calls `engine.evaluate(meta)`** with `meta: VideoMeta` ? confirmed (telethon\_client.py:257).
2. **FilterEngine.evaluate` signature is `(self, meta: VideoMeta)`** ? confirmed (filters/\_\_init\_\_.py:35); VideoMeta has only duration/width/height/size/mime/is\_gif/is\_round/nosound ? no ContentItem/Source/Policy.
3. **Handler builds `VideoRecord` and calls `insert\_video`** ? confirmed (telethon\_client.py:258-271).
4. **Handler never constructs ContentItem, never holds a Source object, never loads a Policy** ?

confirmed by imports (line 38) and grep.

5. `**PolicyContext` (ADR "Stage interface") requires all three: ``content_item: ContentItem`,  
`source: Source`, `policy: Policy`` ? confirmed in ADR.

6. `**F1` declares dependencies "None" and Tier 1 says "extends today's FilterEngine" ? confirmed in ADR phasing table and pipeline diagram.

Now assessing the counter-arguments for refutation. Could the ADR's other sections already cover this migration? The ADR's "Schema ? minimal changes" and F1 scope ("wire into live-message handler") do gesture at integration, but nowhere does the ADR acknowledge that the handler currently produces ``VideoRecord`/`VideoMeta`` and would need to produce ``ContentItem`` + resolve ``Source`` + load ``Policy`` to assemble a ``PolicyContext``. The ``insert_content_item`` path exists in `db.py` but the live handler does not use it ? it still uses ``insert_video``. So the migration gap is real and unstated, and "dependencies: None" genuinely understates it.

This is a real design gap, correctly severity medium (not high ? it doesn't make a false claim that breaks a phase or causes data loss; it understates integration cost and omits a dependency). The finding holds.

#### **19. user (2026-07-06T19:19:01.857Z)**

Structured output provided successfully